

Target-Directed Relational Closure

From recurrence guessing to checkable
nonlinear and behavioral proofs

Prepared for Vladimir Reshetnikov

15 September 2026

The proposed capability. Given a polynomial equality about a recursive computation, construct the additional equations needed to make an induction close. Discover those equations by pulling the target backward through program steps, and stop when their *ideal*, rather than just their linear span, becomes stable. Return explicit polynomial identities, or an exact violating execution.

Delivered	Two executable search algorithms, an independent standard-library checker, retained problems and evidence, regression tests, Lean replay sources, and this article.
Executed	62 synthetic problems under two search policies; 108 evidence files independently accepted; 40 regression tests, including an 80-instance scalar differential check.
Not established	An installed Lean tactic, Lean-kernel acceptance of the new files, or a measured improvement over <code>grind</code> or other Lean tactics.

An incremental research proposal, not a replacement for Forge’s existing architecture. The mathematical foundations are attributed to prior work on weighted automata, polynomial automata, and algebraic invariants.

Abstract

Forge already proposes induction planning, invariant synthesis, theorem-directed search, and exact algebraic certificates. Repeating those proposals would not make it more capable. This article instead fills a specific algorithmic gap: *constructing a mutually supporting family of polynomial invariants when no useful equation is individually conserved*. The extension has a fast linear-span lane and a stronger polynomial-ideal lane. Both start from the requested observation, propagate it backward through total polynomial transitions, and synthesize an inductive relation. The ideal lane avoids the bilinear problem of guessing invariant coefficients and their preservation multipliers simultaneously. Its unbounded version terminates for a precisely specified finite-control fragment by the ascending-chain condition; its bounded implementation can return unknown. Failed initial identities produce concrete rational executions through a coefficient-provenance argument, not merely algebraic countermodels.

A synchronous product turns the same mechanism into an equivalence prover for polynomial folds. This supplies a concrete new behavioral-proof lane for Leant and a verification service for Djex-generated candidates without changing their type-synthesis engines. A SymPy search implementation, a separately written standard-library checker, a 62-problem matched synthetic comparison, 40 regression tests, and 254 generated Lean identity obligations accompany the article. The ideal lane resolves all 62 synthetic problems; the linear lane returns unknown on 16 nonlinear examples. These results establish a tested algorithmic extension, not a Lean tactic comparison. All new Lean files remain explicitly uncompiled in this environment.

Contents

1	The contribution relative to the repository	2
2	A running example that separates the two algorithms	3
3	The exact fragment and its semantics	4
4	The certificate language	5
5	Algorithm I: target-directed linear closure	6
6	Algorithm II: polynomial-ideal closure	8
7	Exact counterexamples from coefficient provenance	10
8	A genuinely coupled nonlinear invariant	11
9	Relational products and whole-fold proofs	12
10	A concrete new lane for Leant and Djex	13
11	Implementation: algorithms, modules, and libraries	14
12	Lean lowering and the remaining verification gap	16
13	Experiments and what their numbers mean	17
14	Integration modules and acceptance gates	19
15	Extensions that reuse the new mechanism	20
16	Limits and conclusions	21
A	Artifact map and reproduction	22
B	Canonical data and exact-check obligations	23
C	Proof obligations for a certified source extractor	24
D	Source and status ledger	24

1 The contribution relative to the repository

1.1 A source-pinned starting point

The inspected Forge revision is 674521027d968d59f7b83220ed52304a85cb55e2. Its merged article already contains the broad proof-planning architecture that this proposal assumes: scoped obligations, explicit induction plans, certificate-producing workers, and a conservative bridge to Lean’s local tactics. The repository explicitly states that Forge is not yet an implemented Lean tactic. Its existing merge records elaboration of three core-only Lean files, not a complete integration or a tactic benchmark [1, 2].

The relevant starting point is more specific than “Forge needs invariants.” The structural chapter implements additive recurrence synthesis and a state-machine format with

$$I(s_0) = 0, \quad I(T(s)) = I(s).$$

It also *already proposes* the more general condition

$$I_i(T(\mathbf{x})) = \sum_j h_{ij}(\mathbf{x}) I_j(\mathbf{x}),$$

and correctly warns that jointly discovering the I_j and h_{ij} is a bilinear search problem. The current `prototype/forge/recurrence.py` implements exact additive and accumulator procedures, rather than the closure algorithm developed here [3, 4].

This article contributes the missing *construction*, its proof obligations, its counterexample extraction, and working experiments. It does not claim that multiple invariants, ideal certificates, or certificate-first automation are new ideas.

Area	Already present in Forge	Addition here
Recursion	Generalization, induction planning, additive polynomial formulas	Target-generated coupled relations, including nonlinear state updates
Algebra	Ideal multipliers in fixed-goal certificates	Automatic construction of a transition-stable target ideal
Search	Typed, scoped, bounded workers	Two concrete closure policies with a shared independently replayable output format
Negative evidence	Exact witnesses and careful verdict boundaries	A coefficient-provenance algorithm yielding an actual violating execution
Behavior	Typed synthesis and proposed higher-order extensions	A relational product reduction for universal polynomial-fold assertions

Table 1: The incremental contribution. The middle column is prior material, not a new proposal.

1.2 What was, and was not, reviewed

The review targeted Forge’s README, the architecture, structural, nonlinear, and integration source sections, its Leant/Djex review, the prototype module inventory, and the current recurrence implementation. It also inspected the Leant and Djex READMEs and Leant’s behavioral-synthesis contract. This is not a claim to have read every line of all nine frozen submissions. A search index returning no matches is not evidence of universal absence. The novelty claim is therefore deliberately scoped: this is a concrete extension beyond the reviewed merged design and its present recurrence worker.

The neighboring revisions are Leant `6bf05ad78c467989e68290f2d08bbbed40802d485` and Djex `e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5`. Leant’s reported dependency pin is separately `e237e866`; the tip of a repository and a downstream dependency pin are not interchangeable [7, 9].

1.3 Why this is worth adding

The capability is useful when the proof obligation contains an unbounded iteration but each *step* is algebraically simple. Examples include two implementations of a numerical fold, a cached intermediate value maintained by a loop, an affine state-space realization, or a polynomial relationship between several accumulators. Local algebra can prove the step once the correct relation is supplied. The difficulty is discovering that relation.

The proposed worker does not search for an arbitrary closed form of the entire computation. It searches only for equations needed to justify the requested observation. That distinction allows it to prove equivalence even when neither side has a small or convenient closed form.

2 A running example that separates the two algorithms

Consider a list of rational inputs $u_1, \dots, u_m$. The first implementation keeps one value z and updates

$$z' = z^2 + u. \quad (1)$$

The second keeps a pair (x, y) and updates

$$x' = y + u, \quad y' = (y + u)^2. \quad (2)$$

Initialize them at $z = x = t$, $y = t^2$. The desired theorem says that after every finite input list, the first result equals the first component of the second. Calling the implementations “slow” and “fast” in the accompanying code is only a mnemonic; no runtime advantage is asserted.

One-step comparison of the visible outputs gives

$$z' - x' = z^2 - y.$$

The original equation $z - x = 0$ is not sufficient by itself. The missing relationship is the meaning of the second accumulator:

$$g_1 = x^2 - y, \quad g_2 = z - x.$$

Their transition identities are

$$g_1(F(z, x, y, u)) = 0, \quad (3)$$

$$g_2(F(z, x, y, u)) = g_1 + (z + x)g_2. \quad (4)$$

At initialization both vanish. These two identities prove the result by induction on the input list. Neither a closed form for z_m nor enumeration of lists is needed.

The conceptual shift. Do not insist that an equation retain the same value after a step. Require only that the *common zero set of a finite family of equations* be preserved. The equations may transform into polynomial combinations of one another.

2.1 Why a linear-only repair is fundamentally weaker

Let T_0 be the product update with input $u = 0$ and let $p = z - x$. For $k \geq 1$,

$$p \circ T_0^k = z^{2^k} - y^{2^{k-1}}.$$

These polynomials have pairwise distinct highest powers of z . They are linearly independent over $\mathbb{Q}$: in a finite linear combination, the coefficient of the largest such power forces its multiplier to be zero, and descending induction forces the rest to be zero.

Thus the vector space generated by repeated pullbacks of p is infinite-dimensional. Increasing a fixed linear-span budget cannot make the underlying space finite. In contrast, the ideal generated by g_1 and g_2 is already stable by (3)–(4). The ideal lane is therefore not merely the same search with a larger budget; it uses a strictly stronger notion of redundant obligation.

2.2 A mutation with an exact witness

Change only the second update in (2) to

$$y' = (y + u)^2 + 1.$$

With $t = 0$ and the two inputs $[0, 0]$, the first implementation ends at $z = 0$; the second ends with first component $x = 1$. The difference is -1 . The prototype discovers this two-step execution and the independent checker recomputes it from the original transition system. A failed polynomial identity is not automatically reported as a counterexample: the actual execution is part of the evidence.

3 The exact fragment and its semantics

Definition 3.1 (Polynomial transition system). A problem consists of a finite set of control locations L , a state vector $\mathbf{x} \in \mathbb{Q}^n$, and a finite set of edges. An edge $e : \ell \rightarrow m$ has a fresh input vector $\mathbf{u} \in \mathbb{Q}^{k_e}$ and a simultaneous polynomial update

$$F_e : \mathbb{Q}^n \times \mathbb{Q}^{k_e} \longrightarrow \mathbb{Q}^n.$$

Each initial branch specifies a location ℓ_b and a polynomial map $I_b : \mathbb{Q}^r \rightarrow \mathbb{Q}^n$. Each target specifies a location ℓ_a and a polynomial $p_a \in \mathbb{Q}[\mathbf{x}]$ whose required value is zero.

A legal execution chooses an initial branch and rational parameter values, then follows any finite sequence of control-compatible edges, choosing arbitrary fresh rational inputs on each edge. The safety property is that every target polynomial vanishes whenever its observation location is reached, including an initial state at that location.

3.1 Deliberate restrictions

All transitions in this first fragment are total. There are no implicit guards, truncated natural subtraction, field divisions by state-dependent expressions, integer remainder operations, floating-point rounds, or partial functions. Initial conditions are given by polynomial parametrizations, not arbitrary logical predicates. There are finitely many locations and state coordinates. Fresh inputs are independent between steps, but occurrences of the *same* input within one step are correlated.

These restrictions are part of the theorem, not parser inconveniences to be silently erased. The fragment includes nonlinear updates of arbitrary degree, but it does not include arbitrary recursive Lean programs.

The implementation uses a uniform state dimension at all locations; unused coordinates can be padded. A future typed implementation may use a dependent state family, but then each edge carries the necessary source-to-destination coordinate correspondence. Nothing in the proof permits an accidental variable permutation.

3.2 Control and nondeterminism

Finite control is not cosmetic. A temporary discrepancy may be legal at an intermediate location and must not become an obligation there merely because it is forbidden at the exit. For example,

$$\begin{aligned} 0 \rightarrow 1 : (x, y) &\mapsto (x + u, y + u + c), \\ 1 \rightarrow 0 : (x, y) &\mapsto (x + v, y + v - c), \end{aligned}$$

with initialization (t, t) at location 0, preserves $y - x = 0$ at location 0 and $y - x - c = 0$ at location 1. The two locations require different equations.

The semantics permits all outgoing edges. That models genuine nondeterministic choice. Forgetting the conditions on a guarded source program also gives such a system, but only as an *overapproximation*. Positive proofs can sometimes transport back through a proved simulation; abstract counterexamples cannot be reported as source counterexamples without checking the guards.

4 The certificate language

For every location ℓ , the certificate supplies a finite list

$$G_\ell = (g_{\ell,1}, \dots, g_{\ell,d_\ell}) \subseteq \mathbb{Q}[\mathbf{x}].$$

Empty lists are allowed. Define

$$\mathcal{I}_\ell(\mathbf{x}) \iff \bigwedge_{j=1}^{d_\ell} g_{\ell,j}(\mathbf{x}) = 0.$$

The following three families of *polynomial identities* constitute the entire positive certificate.

Initialization. For every initial branch b and every generator at its location,

$$g_{\ell_b,j}(I_b(\mathbf{t})) = 0 \quad \text{in } \mathbb{Q}[\mathbf{t}]. \tag{5}$$

Observation. For every target a , provide polynomial multipliers $a_{a,j}(\mathbf{x})$ such that

$$p_a(\mathbf{x}) = \sum_j a_{a,j}(\mathbf{x}) g_{\ell_a,j}(\mathbf{x}). \tag{6}$$

Transition. For every edge $e : \ell \rightarrow m$ and every destination generator, provide $h_{e,i,j} \in \mathbb{Q}[\mathbf{x}, \mathbf{u}]$ satisfying

$$g_{m,i}(F_e(\mathbf{x}, \mathbf{u})) = \sum_j h_{e,i,j}(\mathbf{x}, \mathbf{u}) g_{\ell,j}(\mathbf{x}). \quad (7)$$

The matrix has d_m rows and d_ℓ columns. Swapping those dimensions is a semantic error, not merely a storage convention.

Theorem 4.1 (Certificate soundness). *If (5), (6), and (7) hold, every legal finite execution satisfies all its target observations.*

Proof. Initialization follows by evaluating (5) at the selected parameter values. Suppose $\mathcal{I}_\ell(\mathbf{x})$ holds before an edge $e : \ell \rightarrow m$. Every summand on the right of (7) has a zero generator factor. Therefore every destination generator vanishes at the updated state, so $\mathcal{I}_m(F_e(\mathbf{x}, \mathbf{u}))$ holds. Induction on the finite execution length establishes the invariant at every reached location. At an observation point, (6) evaluates to a sum of zero terms. $\square$

4.1 What the checker does not need

The checker does not need to know how the basis was discovered. It need not verify that a basis is reduced, minimal, a Gröbner basis, or even a basis of the ideal generated by the originally requested targets. It checks the three identity families directly against the original problem. An arbitrary family of equations is acceptable when it passes all three checks.

This removes a substantial proof burden. The search may use Gröbner bases, but accepting the final inductive relation does not require formalizing the Gröbner algorithm. In contrast, a tool whose conclusion specifically asserts that a set is a Gröbner basis must establish that additional fact. Existing Lean polynomial tactics offer certificate-based ideal membership and related results; they are possible implementation dependencies, not the source of a new trust principle here [13].

4.2 Coefficient fields and transport

The executed checker interprets the problem over $\mathbb{Q}$. The positive certificate argument also works after interpreting its rational coefficients in a characteristic-zero field, including $\mathbb{R}$. This is a mathematical transport statement, not evidence that a Lean reifier for those fields was implemented. For an arbitrary commutative ring, rational literals may not make sense. Denominator clearing requires an explicit, source-correct argument. A worker must never reuse the same serialized coefficient interpretation indiscriminately across rings, naturals, finite fields, and machine arithmetic.

5 Algorithm I: target-directed linear closure

5.1 Pull an observation backward

Suppose q is a destination equation for an edge $e : \ell \rightarrow m$. Substitute the update and expand in the fresh input variables:

$$q(F_e(\mathbf{x}, \mathbf{u})) = \sum_\alpha q_{e,\alpha}(\mathbf{x}) \mathbf{u}^\alpha. \quad (8)$$

The coefficients are new possible source equations. The linear lane maintains at each location the vector space spanned by its discovered coefficients. It adds a coefficient only when exact rational linear algebra shows that it is outside the current span.

```

linear_closure(problem):
  seed each location with its target polynomials
  queue := independent seeds, with provenance
  while queue is nonempty:
    take destination generator q at location m
    for every incoming edge e : l -> m:
      image := simultaneous_substitution(q, e.update)
      for coefficient r of image in e.fresh_inputs:
        if r is outside span_Q(generators[l]):
          add r, record its parent coefficient, and enqueue it
          if r(initial_parameters) is nonzero:
            construct and replay an exact execution witness
  reconstruct all base, target, and transition identities

```

Every new independent polynomial is processed once. Span membership returns its exact coordinates when it succeeds; no tolerance appears in that decision. Input-dependent transition multipliers are reconstructed by combining those coordinates with the corresponding input monomials.

5.2 The finite-dimensional guarantee

Proposition 5.1 (Finite observable space). *Suppose that, at every location, all target coefficients and their iterated pullbacks lie in a known finite-dimensional rational vector space V_ℓ . Without resource truncation, linear closure terminates after at most $\sum_\ell \dim V_\ell$ independent insertions and either constructs a positive certificate or exposes a nonzero initial identity.*

Proof. Each insertion strictly increases a subspace dimension at one location, and those dimensions are bounded by the stated sum. Each inserted polynomial has only finitely many incoming edges and input coefficients, so the worklist is finite. At exhaustion every generator pullback has its coefficients in the corresponding source span, which supplies (7). Targets were seeded; initial identities are checked when generators are added. $\square$

For affine targets and updates affine in the state, the affine observable space has dimension at most $n + 1$ at each location. Coefficients of the affine updates may depend polynomially on fresh inputs without breaking this bound. The usual linear-algebraic view of weighted-automaton equivalence is the relevant prior foundation [10].

For nonlinear observables with degree cap D , the ambient dimension is at most

$$M = \binom{n + D}{D}.$$

A bounded implementation must still check that newly generated polynomials fit that cap. Discarding higher-degree terms would be unsound. The bound limits the number of admitted independent equations; it does not guarantee that all pullbacks stay in the space.

5.3 Implementation and cost

The supplied prototype uses exact coefficient matrices and `Matrix.gauss_jordan_solve`. A practical native worker should maintain an incremental row-echelon basis, reducing a new coefficient against

existing pivot rows instead of resolving every span query from scratch. The proof payload retains the reduction coordinates.

The algebraic-operation count depends on the number of monomials, independent rows, edges, and distinct input coefficients. Rational bit lengths can grow substantially, so polynomially many field operations are not automatically a polynomial bit-complexity claim. Sparse pivot selection and fraction-free or modular reconstruction are search optimizations; every reconstructed rational identity must still be checked exactly.

6 Algorithm II: polynomial-ideal closure

6.1 Replace vector-space redundancy by ideal redundancy

In the running example, the coefficient $z^4 - y^2$ is not a constant linear combination of $z - x$ and $z^2 - y$. It is nevertheless an immediate polynomial multiple:

$$z^4 - y^2 = (z^2 + y)(z^2 - y).$$

The ideal lane regards this obligation as redundant. At each location maintain an ideal $J_\ell \subseteq \mathbb{Q}[\mathbf{x}]$ instead of a vector space. A new pullback coefficient is added only when it is not already in that ideal.

```
ideal_closure(problem):
  J[l] := ideal generated by targets at l
  pending := raw target generators with provenance
  while pending is nonempty:
    pop a raw generator q at destination m
    for each incoming e : l -> m:
      expand q(F_e(x,u)) by monomials in u
      for each coefficient r(x):
        if normal_form(r, GroebnerBasis(J[l])) != 0:
          append the ORIGINAL r to J[l]
          enqueue r with coefficient/edge/parent provenance
          invalidate only the basis cache for l
          check r against each initial parametrization at l
          if an initial identity fails: reconstruct a real trace
      choose finite final generators G[l]
    compute quotient multipliers for targets and every edge
    emit identities; let the independent checker decide acceptance
```

The search uses a Gröbner normal form for membership, but retains the *original coefficient* as the newly added raw generator. Storing only the normal-form remainder would lose the simple path provenance used for negative witnesses. A production implementation may retain both, together with the membership decomposition connecting them.

6.2 Why coefficients are the right interface

Lemma 6.1 (Extension by fresh variables). *For an ideal $J \subseteq \mathbb{Q}[\mathbf{x}]$ and $q(\mathbf{x}, \mathbf{u}) = \sum_{\alpha} q_{\alpha}(\mathbf{x}) \mathbf{u}^{\alpha}$,*

$$q \in J \mathbb{Q}[\mathbf{x}, \mathbf{u}] \iff q_{\alpha} \in J \text{ for every } \alpha.$$

Proof. If all coefficients belong to J , expand each as a finite sum of generator multiples and collect

the input monomials. Conversely, in a representation $q = \sum_i a_i(\mathbf{x}, \mathbf{u})g_i(\mathbf{x})$ with $g_i \in J$, comparing each input coefficient expresses q_α as a sum of polynomial multiples of the g_i . $\square$

No finite set of tested input values is used to infer this membership. The operation is a symbolic coefficient extraction over independent indeterminates.

6.3 Why an old generator need not be processed again

Assume a generator's image was already shown to belong to the source ideal extended by the input variables. Enlarging that source ideal cannot invalidate membership. Enlarging a destination ideal by adding a new generator requires processing the new generator, not revisiting the old ones.

More generally, if each current destination generator has a valid image, then for every polynomial combination $q = \sum_i a_i(\mathbf{x})g_i(\mathbf{x})$,

$$q(F_e) = \sum_i a_i(F_e)g_i(F_e)$$

also belongs to the extended source ideal. This is why it suffices to process a finite generating set rather than all its infinitely many multiples.

6.4 A precise termination theorem

Theorem 6.2 (Unbounded target-ideal closure). *For the fragment in Section 3, exact ideal closure without resource bounds terminates. If the property is true, it returns a positive certificate. If the property is false, it exposes a nonzero initial identity from which a finite rational counterexample can be reconstructed.*

Proof. Every insertion strictly increases the ideal at one location. A polynomial ring in finitely many variables over a field is Noetherian, so it has no infinite strictly ascending chain of ideals. With finitely many locations, infinitely many insertions would force an infinite ascending chain at some location. Therefore only finitely many insertions occur. Each insertion generates only finitely many worklist operations and coefficient polynomials. Exact Gröbner membership terminates on each finite input, so the unbounded worklist terminates.

If no initial identity fails, every raw generator vanishes under its initial parametrizations. At worklist exhaustion, all generator pullbacks are in the respective source ideals extended by fresh input variables. A finite final basis therefore admits all the identities of Section 4; Theorem 4.1 proves the property.

If the property is false, this positive conclusion is impossible, so some initial identity must fail before such a certificate is completed. The witness construction of Section 7 converts its retained coefficient provenance into a finite execution violating an original target. $\square$

The proof deliberately states the exact fragment. It is not a theorem about arbitrary guarded imperative programs, arbitrary Lean recursion, or generating all polynomial invariants. It verifies *one finite collection of given polynomial observations*. Connections between polynomial automata, zeroness, and algebraic invariant computation are established research, not a discovery claimed by this article [11, 12].

6.5 Why this avoids the bilinear template problem

The algorithm never solves simultaneously for unknown coefficients of g_j and h_{ij} . At each step, the requested observation and the program determine a concrete next polynomial. Ideal membership is asked relative to a concrete finite generator list. Once the list stabilizes, quotient multipliers are computed for fixed polynomials.

The tradeoff is important: replacing a bilinear template search does not make all resulting computations cheap. Gröbner bases, substitution, coefficient expansion, and ascending-chain length can be very expensive. The benefit is a well-defined complete unbounded procedure for a useful fragment and a small certificate boundary, not a general polynomial-time algorithm.

6.6 Basis choice and certificate size

The executable prototype stores raw generators for discovery but emits a reduced Gröbner basis for replay. This makes quotient extraction convenient, because SymPy reduces against that basis directly. It need not produce the smallest certificate. The coupled example in Section 8 has two natural generators but the chosen reduced basis has three.

A useful optimization is to retain the transformation from the original requested generators to the Gröbner basis, and lift the quotient vectors back to the original small family. Singular’s `liftstd` and `lift` are concrete operations to investigate for this purpose [15]. This optimization is proposed, not implemented in the delivered prototype. Its result must satisfy the same identity checker; the backend’s preferred basis is not authoritative.

7 Exact counterexamples from coefficient provenance

A nonzero initial polynomial is evidence that the proposed invariant family fails somewhere, but an arbitrary family could simply have been too strong. The closure construction supplies additional provenance that makes the failure meaningful for the *original target*.

Every raw generator is either a target or one coefficient of a pullback of an earlier generator along an incoming edge. Each node stores its parent, the edge, and the chosen input monomial. Suppose a generated coefficient r at a source state s is nonzero, where

$$q(F_e(s, \mathbf{u})) = \sum_{\alpha} r_{\alpha}(s) \mathbf{u}^{\alpha}$$

and $r = r_{\beta}$. The right side is then a nonzero polynomial in $\mathbf{u}$. Consequently some rational input makes q nonzero in the successor state. Following the parent pointer repeats this argument until an original target is nonzero.

Lemma 7.1 (Deterministic interpolation grid). *Let $p \in \mathbb{Q}[t_1, \dots, t_r]$ be nonzero, with individual degree at most d_i in t_i . There is a point in $\prod_i \{0, 1, \dots, d_i\}$ where p is nonzero.*

Proof. For one variable, a nonzero polynomial of degree at most d cannot have $d + 1$ distinct roots. For several variables, write p as a polynomial in its last variable. If it vanished at every grid point, fixing the preceding coordinates and using the one-variable result would make every coefficient vanish on the smaller grid. Induction would make all coefficient polynomials zero, a contradiction. $\square$

Proposition 7.2 (Provenance yields a concrete witness). *If a raw closure generator has a nonzero polynomial pullback through an initial parametrization, its provenance yields a finite control-compatible execution, with rational parameters and inputs, on which an original target is nonzero.*

Proof. Use Lemma 7.1 on the failed initial identity to obtain parameters and an initial state where the generator is nonzero. If the node is not a target, its parent pullback has a nonzero coefficient at that state. Apply the same lemma to find inputs making the parent nonzero after the recorded edge. The edge is directed from the child node’s location to the parent’s location, so the resulting execution follows control flow. Parent pointers lead to strictly earlier discovery nodes and hence terminate. At the root, the nonzero polynomial is an original target at the reached observation location. $\square$

7.1 The order of the reconstructed execution

Discovery proceeds backward from an observation, but the witness runs forward from initialization. A provenance chain must therefore be interpreted in its source-to-parent order, not copied as the order in which edges happened to be processed. The regression corpus includes a three-location, two-edge system whose transitions do not commute. Reversing the witness edges is rejected by the checker.

7.2 A false statement that finite samples miss

Consider the initial state $x = 0$ and transition

$$x' = x + u(u - 1)(u - 2), \quad \text{target } x = 0.$$

For every finite execution using only $u \in \{0, 1, 2\}$, the target holds. Thus a large sample of long lists over that alphabet can be entirely misleading. Coefficient extraction immediately sees a nonzero coefficient, and the grid construction finds $u = 3$, giving $x' = 6$. The retained evidence records that one-step trace. A separate test restricts the witness back to each of 0, 1, 2 and confirms that these alleged counterexamples are rejected.

The grid can be exponentially large in the number of fresh inputs. The implementation charges each grid point and returns unknown on exhaustion. A future solver may propose a smaller witness, but it cannot replace exact replay of the source transition system.

8 A genuinely coupled nonlinear invariant

Let the initial state be $(x, y, z) = (t, t^2, t^3)$ and let

$$\begin{aligned} x' &= x + u, \\ y' &= (x + u)^2 + (z - x^3), \\ z' &= (x + u)^3 + (x + u + c)(y - x^2), \end{aligned}$$

where c is a fixed rational constant. The only requested observation is $y = x^2$. Starting from $g_1 = y - x^2$, one pullback produces

$$g_1(F) = z - x^3 =: g_2.$$

The new equation is not an arbitrary template guess; it is demanded by the step case. Pulling it back gives

$$g_2(F) = (x + u + c)g_1.$$

The resulting certificate is the matrix identity

$$\begin{pmatrix} g_1(F) \\ g_2(F) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ x + u + c & 0 \end{pmatrix} \begin{pmatrix} g_1 \\ g_2 \end{pmatrix}.$$

Both generators vanish at initialization. Hence the requested observation holds for every finite input list. Except at special states, neither generator is individually conserved. The product multiplier $(x + u + c)$ explains why constant linear-span closure can keep adding equations such as xg_1, xg_2 , and higher polynomial multiples.

The corpus contains eight instances with $c = 1, \dots, 8$. With the same degree cap of six, the ideal lane proves all eight; the linear lane returns unknown on all eight. The machine certificate’s reduced basis is not the explanatory two-generator basis above, and the result records distinguish *raw independent insertions* from *final replay basis size*.

9 Relational products and whole-fold proofs

9.1 The product construction

Suppose two implementations consume the same input list and have state updates $A(s, u)$ and $B(t, u)$. Their synchronous product is

$$F((s, t), u) = (A(s, u), B(t, u)).$$

If their outputs are polynomial observations $o_A(s)$ and $o_B(t)$, ask the closure worker to prove

$$p(s, t) = o_A(s) - o_B(t) = 0.$$

The initial product state preserves any shared parameters and dependencies. Theorem 4.1 then proves equality of the observations for every finite input list, not merely agreement on a finite behavior table.

This is a reduction, not an assumption that all recursive functions are folds. The source extractor must prove that each function is represented by the claimed transition. A term can be mathematically equivalent to a fold without being recognized by the first implementation.

9.2 Shared inputs must really be shared

A common translation error is to give the two sides distinct fresh inputs. For updates $s' = s + u$ and $t' = t + u$, equality is preserved. For the erroneously translated product $s' = s + u$, $t' = t + v$, it is not. The shared input is one binder in the product’s source-owned variable table. Reifying each side independently and concatenating variable names is insufficient.

The same discipline applies to initial parameters. Two uses of one source parameter must become one neutral parameter; two distinct binders must not be merged because they print the same name. This uses Forge’s existing identity boundary rather than proposing a second identity scheme [6].

9.3 Source forms for a first extractor

The first useful extractor should recognize an explicit `List.foldl` with a fixed-size product of rational state coordinates and a polynomial step. Accept projections, tuples, rational constants, addition, subtraction, multiplication, and fixed natural powers. Allow a finite sum of input tags by compiling each tag to a control-compatible edge with its payload coordinates. Use exact source equations to normalize a limited number of wrappers.

A second extractor may recognize structurally recursive list functions whose recursive calls all consume the tail and whose nonstructural arguments update polynomially. It must prove the accumulator/fold representation before asking the closure worker about it. General dependent recursion, well-founded recursion with arithmetic stopping conditions, and higher-order callbacks with unmodeled behavior stay outside this first contract.

9.4 State summaries without promising full equivalence

The discovered family can be useful even when it proves only a selected observation. For example, equal checksums are not equal lists. A summary certificate may justify that checksum assertion and still be insufficient to deduplicate the underlying candidate programs.

A proposed later optimization is to use the finite linear observable space as a quotient of a state machine for the selected observations. Such a quotient can reduce repeated searches. It must retain a simulation proof and an explicit observation interface. The delivered code constructs closure certificates, not a general semantic equivalence relation on Djex terms.

10 A concrete new lane for Leant and Djex

Leant’s named behavioral entrance first checks the exact candidate’s type and then tries proofs of the supplied proposition and its negation by `decide` and bounded `simp`. Its contract explicitly distinguishes passed, falsified, and inconclusive assertions. Quantified properties can remain inconclusive when these proof attempts do not suffice [8].

The new lane should be inserted *after exact candidate elaboration*, as a bounded additional discharge method for supported universal fold assertions. It should not modify Djinn’s inhabitation verdicts or Exference’s typeclass resolution. A candidate remains tied to its original source graph and exact rendered occurrence.

10.1 Proposed assertion-discharge protocol

```
discharge_polynomial_behavior(exactCandidate, assertion, budget):
  recognize assertion as a supported fold observation
  extract each fold into a typed transition view
  prove source-to-transition correspondence
  form a synchronous product when two computations are compared
  try linear closure under its quota
  if still unknown, try ideal closure under a separate quota
  replay returned polynomial identities in Lean
  apply fold/path induction and source correspondence
  return a proof of THIS assertion about THIS candidate
```

The phrase “prove source-to-transition correspondence” is an implementation obligation, not a step delegated to an untrusted parser. If it fails, the lane returns unsupported or unknown. For a checked abstract counterexample, the implementation reconstructs the original input list, evaluates the candidate’s actual source semantics, and proves the negation of the precise assertion before labeling that source candidate falsified.

10.2 Why this helps without changing type synthesis

A type such as `ListRat→Rat` admits many implementations. Type-directed synthesis alone does not distinguish the desired recurrence from an unrelated one. A finite assertion table may distinguish some candidates but cannot prove a universal law. The proposed lane can certify all-list behavior for its fragment using finitely many algebraic identities.

It can also expose a useful synthesis feedback signal. A checked counterexample is a concrete list and initial parameter assignment. Leant may add that example to its existing candidate-rejection workload, preserving all original budgets. The example rejects only candidates independently shown to violate it. A failed closure attempt does not authorize pruning a term or a source specialization.

10.3 A compositional candidate policy

For a candidate whose typed graph visibly contains two polynomial folds, construct the corresponding paired obligation without asking the synthesis engine to discover an explicit relational proof term first. This separates *constructing the program* from *discovering the induction relation*. It is a concrete way for Forge to add capability to Leant rather than copying its term-synthesis machinery.

The currently reported integer-indexing and simultaneous two-universe composition gaps in Leant/Djex are not solved by these experiments. Neither is arbitrary Church-encoded behavior. The proposed first integration targets exact, source-visible polynomial folds. The type-synthesis libraries remain useful for candidate generation and source ownership, but no Haskell value is treated as a Lean proof merely because Djex accepts its type [7, 9].

11 Implementation: algorithms, modules, and libraries

11.1 What the executable prototype actually contains

`prototype/search.py` is an untrusted proposer. It uses SymPy 1.14.0 for simultaneous substitution, exact polynomial expansion, coefficient extraction, rational linear systems, Gröbner bases in graded reverse lexicographic order, and quotient/remainder computation. The exercised entry points are `Poly`, `Matrix.gauss_jordan_solve`, `groebner`, and the basis's `reduce` method [14].

`prototype/verify_stdlib.py` is a separately written checker. It imports no search code and no SymPy. It has its own sparse polynomial operations over `fractions.Fraction`. The proposer and checker share a documented JSON schema, not an algebra implementation. The checker recomputes initial substitutions, every target combination, and every transition identity against a separately supplied original problem. Counterexample replay recomputes all states; states supplied by the search would not be trusted.

`prototype/cases.py` builds the deterministic synthetic corpus. `prototype/run_experiments.py` executes both policies and checks their outputs before recording acceptance. `prototype/replay_corpus.py` replays retained evidence with Python site packages disabled. The test suite exercises search cutoffs, arithmetic semantics, schema rejection, witness reconstruction, and independently evaluated scalar transitions.

11.2 Library choices for a production implementation

Component	Concrete choice	Role and acceptance condition
Reference search	SymPy <code>Poly</code> , rational matrices, <code>groebner</code> , <code>reduce</code>	Already exercised. Retain as a readable reference and a cross-check for optimized workers.
Larger ideal searches	Singular, optionally through SageMath	Use exact standard bases and lifting matrices. Recheck every emitted coefficient identity; no backend status is a theorem.
Small Lean replay	Mathlib <code>ring</code> , <code>norm_num</code> , and ordinary induction	Emit concrete identity proofs first. This is a modest bridge before a reflective checker. New emitted files are uncompiled here.
Polynomial Lean infrastructure	<code>MvPolynomial</code> ; evaluate WuProver’s <code>GroebnerTactic</code> and <code>MonomialOrderedPolynomial</code>	Existing formal algebra and certificate-oriented tactics are relevant. Their compatibility with Forge’s pinned project must be tested, not assumed.
Typed extraction	Lean metaprogramming and, where appropriate, <code>quote4</code>	Preserve exact source expressions while constructing a polynomial view and its correctness theorem. Extraction is proposed, not implemented in this package.
Candidate source	Djex’s curated <code>Language.Haskell.Djex</code> or focused <code>Language.Haskell.Synthesis.*</code> interfaces	Keep the candidate’s own source evidence. Do not build a new adapter on accidental internal-module APIs.

Singular and SageMath expose the needed ideal/standard-basis operations in their documentation [15, 16]. The recently described `GroebnerTactic` project provides concrete Lean/CAS certificate machinery for related polynomial tasks; adopting it would require checking its exact imports, toolchain, and trust path against the Forge pin [13]. No build or speed claim for those optional dependencies is made here.

11.3 Why the first version should not use a floating solver

This fragment is exact by construction. A floating linear or nonlinear optimizer would introduce rational reconstruction problems without being necessary for the reference algorithm. Search can still use numerical or modular heuristics later to suggest supports or pivots, but the prototype’s actual membership and identity computations use rational coefficients throughout.

11.4 Budget semantics

The recorded run uses maximum generated state degree six, 40 raw independent generators, 200 pullbacks, 1,500 terms per candidate coefficient, and 100,000 witness-grid evaluations. A ten-second cooperative search deadline is checked between operations. A single CAS call is not cooperatively preempted, so the runner also owns a disposable worker process and applies an external timeout.

The checker has independent caps on input bytes, monomials, rational bit lengths, expanded degree, and arithmetic operations. These are useful resource guards, not a claim that the Python parser is a hardened adversarial service. Expanding a large product or parsing deeply nested JSON can consume resources before a later check notices a limit; process isolation remains necessary.

The prototype checks a generated degree cap before ideal membership. This is conservative but can miss

an easy high-degree member of an existing small ideal. An improved worker can first attempt bounded reduction without full expansion, while preserving the same exact final checker. That optimization must not silently replace a discarded term with zero.

12 Lean lowering and the remaining verification gap

12.1 An implementation sequence that avoids a large trusted checker

The first Lean integration should lower the received basis and multipliers to ordinary rational expressions. For each row, construct a goal asserting the literal identity and close it with `ring`. Evaluate initial identities the same way. Combine them with conjunction introduction and elimination to obtain an invariant-preservation theorem. Finally apply an ordinary fold or reachability induction theorem and the certified source extraction.

The fact that a returned family happens to be a Gröbner basis should not appear in the resulting theorem unless needed elsewhere. A final proof uses only its initial vanishing, preservation identities, and implication of the requested target.

12.2 Delivered Lean artifacts

`lean/Core.lean` contains a generic reachability-invariant theorem and a fold-preservation skeleton. `lean/IntegrationSpecimens.lean` gives an explicit full proof script for the running fold equivalence, its two-generator algebra, the coupled transition identities, and the two-step mutant control. These are integration targets, not a `forge` syntax implementation.

`prototype/emit_lean.py` generates `lean/GeneratedIdentities.lean` from the 45 accepted positive ideal certificates. It emits 254 concrete initial, observation, and transition identity declarations. Emission first replays the JSON certificates with the independent checker. This guards the generator's input, but does not make the emitted text a checked Lean proof.

Lean status: NOT_RUN. No Lean or Lake executable was available in this environment. The attempted compilation runner records this explicitly in `results/lean-status.json`. None of the new Lean declarations is claimed to have elaborated, and no axiom audit was run. The polynomial-identity emitter is not a full compiler from the JSON problem to its original Lean goal.

The supplied Lake configuration pins Lean `v4.34.0` and Mathlib `1cf325a0cf67aca2b04d76b5380ff6a9e410aefa`, matching the Forge baseline. A maintainer should compile the core theorem, then the explicit specimens, then the generated identities, and only afterward integrate the extractor and tactic front end. A compilation failure is a reason to repair the script, not to add `sorry` or relax the assertion.

12.3 A later reflective implementation

When identity replay becomes a measured bottleneck, define a computable sparse polynomial representation in Lean, its evaluation function, and correctness lemmas for addition, multiplication, substitution, and normalization. The certificate-checking theorem then states that a successful Boolean check implies the three identity families under the source interpretation.

This does not require formalizing the search algorithm to trust individual results. A separate formalization of termination or completeness would be valuable research, but it must not delay a small proof-producing integration. Existing computable polynomial work is a better starting point than another unreviewed private algebra implementation [13].

13 Experiments and what their numbers mean

13.1 A matched synthetic comparison

The main corpus contains 62 distinct problem files. Both closure policies run on exactly those files, giving 124 policy runs. Construction parameters and expected outcomes are fixed in `cases.py`; the affine coordinate changes use seed 20260915. These are designed examples, not a sample of Mathlib goals. The two policies share their degree, generator, pullback, and coefficient caps.

Every terminal result is checked before it enters the accepted ledger. Successful search means “a candidate certificate was produced” until that independent replay succeeds. All 16 unknown results are degree-cap outcomes; there is no accepted negative verdict derived from a timeout or failed search.

Family	Cases	Linear closure			Ideal closure		
		P	R	U	P	R	U
Scaled graph	8	8	0	0	8	0	0
Coupled nonlinear	8	0	0	8	8	0	0
Nonlinear equivalence	8	0	0	8	8	0	0
Nonlinear mutant	8	0	8	0	0	8	0
Affine equivalence	12	12	0	0	12	0	0
Affine mutant	6	0	6	0	0	6	0
Control flow	4	4	0	0	4	0	0
Boundary controls	8	5	3	0	5	3	0
Total	62	29	17	16	45	17	0

Table 3: Recorded outcomes. P: positive certificate independently accepted; R: concrete counterexample independently accepted; U: unknown. The total counts are legitimate here because the problem set and policies are explicitly paired. They are not totals of Forge’s nine historical experiments.

The 108 emitted evidence files consist of 74 positive certificates and 34 counterexamples. Many concern the same problem under two different policies; 108 must not be described as 108 distinct solved theorems. The ideal lane’s 45 positive and 17 negative results account for all 62 distinct problems.

13.2 Independent replay

The retained evidence was replayed with

```
python -S prototype/replay_corpus.py --output results/stdlib-replay.json
```

All 108 files were accepted. The receipt records that site packages were disabled and neither `search` nor `sympy` was imported. The run took about 0.044 seconds in this environment, excluding search and the standalone Python process startup. This is a local replay observation, not a Lean checking measurement or a general performance estimate.

The independent checker is not formally verified. Its agreement with search, its different implementation, its exact arithmetic, and its rejection controls make the evidence stronger than re-running the proposer’s own simplifier, but none of them substitutes for Lean-kernel checking.

13.3 Regression and differential tests

Forty named regression tests passed. They exercise changed initial states, changed targets and transitions, missing matrix dimensions, wrong multipliers, malformed rational encodings, duplicate monomials, floats, Boolean exponents, resource exhaustion, shared inputs, simultaneous substitution, unreachable observation points, and reversed witness control flow.

One of those 40 tests contains a separate 80-instance scalar differential experiment. Its transitions have the form

$$x' = ax + bx^2 + cxu + du, \quad x_0 = 0,$$

with fixed-seed small integer coefficients and $d \in \{0, 1\}$. It checks each accepted result and also evaluates all words of length at most three over $\{-1, 0, 1\}$ using the checker’s arithmetic. The 40 cases with $d = 0$ preserve zero; the 40 with $d = 1$ admit a violating input. The finite evaluation is an additional bug detector. It is not the acceptance argument for positive cases.

These 80 instances are not added to the main corpus totals, and the number of assertions inside a test is not presented as a count of Lean proofs.

13.4 Why no speedup is claimed

The runner records discovery time, total worker-process time, checking time, raw generator count, Gröbner calls, and maximum generated degree. Those measurements are useful for debugging. The recorded experiment was resumed after interruption: its early cases ran sequentially and later cases ran with six concurrent workers. Process startup, scheduling contention, and warm-up therefore differ between portions of the run.

Consequently this article does not use the timing columns for a comparative speedup claim. The count comparison remains valid because it uses the same problems, algorithms, and algebraic caps, and the recorded unknown reasons are not timeouts. A future performance study should use a uniform execution policy, repeat runs, report distributions, and include extraction and Lean replay.

An initial runner error duplicated a keyword while building a result row; it was corrected before the accepted ledger was produced. Its failed-run log is retained separately. That diagnostic is neither a solver refutation nor an accepted benchmark result.

13.5 A fair future Lean evaluation

A meaningful end-to-end experiment must use source Lean statements, not just JSON systems. Fix exact theorem files and imports; verify that the intended methods cannot simply use the theorem under test as a premise. Compare ordinary **grind**, an explicit induction-plus-algebra baseline, and Forge with the new worker. Report extraction success separately from search success and final kernel acceptance.

The strongest baseline is not “**grind** with no induction” when the task obviously needs induction. Include a manually supplied correct invariant to measure the cost of discovering the relation rather than confusing it with the cost of checking the step. Count proof size and heartbeats as well as wall time, and preserve all unsupported and unknown cases.

14 Integration modules and acceptance gates

The following are *proposed new paths*, not files asserted to already exist in Forge. They fit the existing worker/obligation boundary rather than replacing it.

Proposed module	Responsibility and acceptance gate
<code>Forge/Relational/System.lean</code>	Typed finite-control polynomial view; source variables and shared inputs have exact ownership. Gate: explicit round-trip examples and rejected arithmetic-domain mismatches.
<code>Forge/Relational/ExtractFold.lean</code>	Recognize supported folds and prove source correspondence. Gate: the original theorem is reconstructed without trusting serialized source text.
<code>Forge/Relational/Product.lean</code>	Pair computations using shared input binders. Gate: correlated-input and initial-parameter tests distinguish intentional sharing from accidental name equality.
<code>Forge/Relational/Certificate.lean</code>	Represent basis, target rows, and edge matrices. Gate: all 254 emitted identities compile, then the path-induction composition closes the corresponding source examples.
<code>Forge/Relational/Replay.lean</code>	Lower identities through <code>ring</code> or a proved sparse checker. Gate: bad initial states, missing rows, and altered multipliers are rejected in Lean.
<code>Forge/Workers/Relational.lean</code>	Route unsupported, linear, and ideal problems under distinct quotas. Gate: cutoff returns unknown; a successful worker adds only a checked proof of its assigned proposition.
<code>forge/relational_closure.py</code>	External reference search in the prototype package, with a canonical data-only protocol. Gate: independent replay and a supervised process timeout; no arbitrary code execution from a request.
Leant assertion adapter	Invoke the worker on supported universal behavior after exact candidate elaboration. Gate: proof and counterexample refer to the same candidate and original assertion, including the final input list.

14.1 Order of implementation

The first milestone is a compiled explicit example: the running fold theorem using a supplied two-generator relation. The next milestone adds JSON certificate replay but keeps the transition system hand-authored. The third adds verified extraction for explicit `List.foldl`. Only then should the external search be invoked automatically from a tactic. Finally, expose the behavioral lane in Leant with its existing candidate provenance and budget accounting.

This order gives each failure a small surface. A search algorithm should not be debugged simultaneously with source extraction, expression lowering, metavariable scope, external process recovery, and proof reconstruction.

14.2 A useful first user-facing syntax

A possible interface is

```
forge (workers := [relational])
forge? (workers := [relational])
```

with optional diagnostics displaying the discovered invariant relation and whether linear or ideal closure was used. These are proposed commands only. The ordinary success result remains a proof of the original goal. A diagnostic counterexample should identify the input list and show a separately checked negation, not mark a false goal as solved.

A trace explanation should be small: requested observation, added invariant components, maximum algebraic degree reached, the preserved relation, and the first unsupported extraction node or exhausted resource. Dumping a Gröbner basis without its relationship to the user’s computation is not sufficient explanation.

15 Extensions that reuse the new mechanism

15.1 Guards through certified local obligations

For a guarded edge, suppose its equality guard is represented by equations $r_k(\mathbf{x}) = 0$. A sufficient preservation identity is

$$g_i(F_e) = \sum_j h_{ij} g_j + \sum_k a_{ik} r_k.$$

The guard terms vanish only while taking that edge. They are not permanent invariants at the source location. For inequality guards, a preservation implication may be discharged by Forge’s existing ordered-algebra certificate worker instead [5].

An initial experimental extension could keep the discovered target family fixed and ask the existing arithmetic workers to prove guarded preservation. This produces a useful positive-only fallback without claiming a new complete guarded search. Counterexamples must replay all original guards; a trace in the unguarded overapproximation is not enough.

15.2 Discover equations before asking for inequalities

Suppose a proof asks for an inequality about several accumulators, but the accumulators have hidden polynomial dependencies. The closure worker can first certify an equality relation for demanded subobservations, and the existing nonlinear worker can use those equations as ideal terms in its positivity certificate. Conversely, a proved algebraic simplification can shrink the state coordinates before closure.

This is a specific cooperation protocol: relational closure publishes a proved relation; ordered algebra consumes it under the same source context. It does not allow the two workers to assume one another’s conjectures. The general cooperation architecture already exists in Forge; this section identifies the new relation-producing worker’s useful outputs rather than restating that architecture.

15.3 Compositional summaries of checked components

A checked fold relation can serve as a summary for a larger program, provided the summary’s parameters, preconditions, and observations are explicit. For a larger synchronous product, a previously checked component relation can reduce the needed state variables or discharge part of the initial relation. The summary is an ordinary theorem, not a cached Boolean result.

The first useful cache should store the certificate inputs and source correspondence needed to replay that theorem. The repository already defines strict identity and scope rules; this extension should reuse them. No new receipt or cache authority is justified by an algebraic basis fingerprint.

15.4 Template synthesis as a later, separate problem

If an update contains unknown program coefficients, closure becomes a different problem: the transition system itself is not fixed. Treating those unknowns as universally quantified parameters would prove a theorem for *every* coefficient choice, not synthesize one. Treating them as unconstrained existential values while extracting input coefficients can also change the quantifier order.

A principled extension can enumerate or solve for a candidate coefficient assignment, freeze it, then run the verified relation checker. It may use counterexamples to refine the assignment search. This is a future program-synthesis layer; the delivered algorithm verifies fixed candidate programs and must not be advertised as automatically solving arbitrary nonlinear sketches.

16 Limits and conclusions

The main limitation is not soundness of the mathematical rule but scale and integration. Polynomial expansion can explode. Ideal chains have a finite termination proof but no small uniform practical bound asserted here. The bounded linear and ideal searches may return unknown on true statements. The source extractor, tactic controller, and complete Lean replay path still need implementation and compilation.

There are semantic limits as well. Equality of selected observations does not imply arbitrary program equivalence. Positive polynomial identities do not supply loop termination. A total-polynomial abstraction does not establish the validity of a guarded counterexample. Rational arithmetic is not interchangeable with natural subtraction, integer division, or fixed-width arithmetic. None of these boundaries should be hidden behind a generic “solver failed” message.

Within those boundaries, the delivered addition is concrete. It constructs the missing induction relation from the goal itself, distinguishes linear from polynomial dependence, emits a small algebraic interface for proof replay, and finds actual violating executions. The paired synthetic experiment shows a clear capability difference between the two algorithms, including a family whose linear pullback space is provably infinite while its ideal certificate has only two explanatory equations.

Recommendation. Add a target-directed relational worker to Forge, starting with explicit rational folds and ordinary Lean identity replay. Keep linear closure as a cheap lane, use ideal closure when polynomial multiples cause linear growth, and make source-certified relational products the first bridge to Leant’s universal behavioral assertions.

A Artifact map and reproduction

Path	Contents
article/	This standalone LaTeX source and compiled PDF.
prototype/search.py	Linear and ideal closure, provenance, and witness construction.
prototype/verify_stdlib.py	Independent exact certificate and execution checker.
prototype/cases.py	The 62-problem deterministic corpus generator.
prototype/test_closure.py	40 regression tests, one containing 80 scalar differential instances.
prototype/run_experiments.py	Paired runner with supervised worker processes and optional exact-record resumption.
prototype/emit_lean.py	Generates 254 identity obligations from 45 positive ideal certificates.
lean/	Generic semantic skeleton, full worked specimens, generated identities, and a pinned Lake configuration; all uncompiled.
results/experiment/	62 original problems, 124 run records, 108 evidence files, CSV measurements, summary, and environment manifest.
results/	Independent replay receipts, unit-test log, Lean status, identity-emission status, and a separated failed-run diagnostic.
docs/	Revision ledger, status, schema and integration notes.

From the archive root:

```
python -m pip install -r requirements.txt
python -m unittest discover -s prototype -p 'test_*.py' -v
python -S prototype/replay_corpus.py
python prototype/run_experiments.py --output reproduced-results --jobs 1
python -S prototype/emit_lean.py
```

For a new canonical JSON problem, use the supervised general-purpose entrance:

```
python prototype/solve_problem.py problem.json --mode ideal --evidence evidence.json
```

It checks the original schema, constructs symbolic expressions without evaluating source strings, and independently replays any returned evidence.

Use `--jobs1` for an uncomplicated timing rerun. The default output directory for experiments is separate from recorded evidence, and nonempty outputs are rejected unless `--resume` is explicit. Resumption checks that existing problem and evidence files match; it is not permission to substitute new problems under old result names.

Once Lean and Mathlib are installed:

```
cd lean
lake update
lake exe cache get
cd ..
python prototype/check_lean.py --output reproduced-lean-status.json
```

These commands are supplied for a future local run. They were not executed here. The runner

distinguishes missing tools, timeout, elaboration failure, and elaboration success. It does not perform a transitive axiom audit.

Build the article with ordinary pdfLaTeX:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge-relational-closure.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-relational-closure.tex
```

No font files, external solver installations, or copies of the source repositories are bundled.

B Canonical data and exact-check obligations

A polynomial is a sorted list of nonzero monomial terms. A term contains an exponent vector and a rational pair of canonical decimal strings:

```
[ [ [exponent_0, ..., exponent_n_minus_1], [numerator, denominator] ], ... ]
```

The denominator is positive, numerator and denominator are coprime, negative zero is forbidden, exponent entries are actual nonnegative integers rather than Booleans, and duplicate monomials are rejected. Zero is an empty term list. Variable order is determined by the separately supplied problem, not a field inside a certificate polynomial.

The original problem schema is

```
{
  "schema": "forge-polynomial-system-v1",
  "name": "informational name",
  "n": state_dimension,
  "locations": location_count,
  "parameters": initial_parameter_count,
  "initial": [{"location": 1, "state": [parameter_polynomials]}],
  "edges": [{"src": 1, "dst": m, "inputs": k,
             "update": [polynomials_in_state_then_inputs]}],
  "targets": [{"location": 1, "polynomial": state_polynomial}]
}
```

The positive certificate contains only

```
{
  "schema": "forge-closure-cert-v1",
  "basis": [basis_polynomials_at_each_location],
  "target": [target_multiplier_rows],
  "step": [edge_matrices_destination_rows_source_columns]
}
```

The certificate cannot choose a replacement transition, target, domain, initialization, or variable dimension. All these are read from the original problem argument.

The negative evidence is an execution request, not a list of trusted states:

```
{
  "schema": "forge-counterexample-v1",
  "initial": initial_branch_index,
  "parameters": [exact_rational_values],
  "steps": [{"edge": edge_index, "inputs": [exact_rational_values]}],
  "target": target_index
}
```

The checker computes the selected initial state, verifies each edge’s source location, evaluates all coordinates of each update using the *old* state, and checks a nonzero original target at the final location. Missing steps, incorrect input arities, wrong locations, and a final zero value are rejected.

C Proof obligations for a certified source extractor

An extraction receipt for a supported fold should contain the exact source step f , the neutral polynomial update F , and a representation map ρ from source states to coordinates. Its essential theorem has the form

$$\rho(f(s, u)) = F(\rho(s), \sigma(u)),$$

where σ represents the source input with its exact domain. Initialization and observation maps need analogous theorems. A product extractor proves that the same $\sigma(u)$ is used by both sides.

For a source invariant goal, positive transport follows by induction on the source list using this commuting-step identity. Negative transport needs a source inhabitant for every selected parameter and input, and an actual source execution. A rational value outside a restricted source input type cannot be used merely because it satisfies the neutral variable’s type.

For a finite-control source representation, the correspondence theorem must also map source control constructors to exact location numbers and preserve which transitions are permitted. This is where guard erasure is recorded as an overapproximation rather than silently made exact.

These obligations state what the proposed Lean module must prove. They are not implemented by the JSON parser or established by Python test counts.

D Source and status ledger

Claim class		Status in this package
Mathematical soundness	certificate	Proved in ordinary mathematics in Theorem 4.1; not a Lean formalization of the checker.
Linear-fragment termination		Proved under finite observable-space hypotheses in Proposition 5.1; bounded searches still admit unknown.
Ideal-fragment termination		Proved for the explicitly total polynomial fragment in Theorem 6.2; no general program-verification claim.
Search implementations		Executed on the stated synthetic suites.
Independent Python replay		Executed with site packages disabled; 108 retained evidence files accepted.

Claim class	Status in this package
Lean identity emission	Executed; 254 declarations generated from 45 positive ideal certificates.
Lean elaboration and kernel checking	Not run; no compiler available.
Forge/Leant/Djex integration	Proposed module and protocol design only; no repository changes or live integration executed.
Comparison with <code>grind</code> , <code>Ae-sop</code> , or other tactics	Not run; no solved-goal or performance claim about those tactics.

References

- [1] Vladimir Reshetnikov, *Forge: README and status*, revision 674521027d968d59f7b83220ed52304a85cb55e2. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/README.md>.
- [2] *Forge*, same revision, `article/sections/03-architecture.tex` and `docs/IDEAS-FROM-LEANT-DJEX.md`. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/03-architecture.tex>.
- [3] *Forge*, same revision, `article/sections/05-structural.tex`, especially the state-machine invariant discussion and the warning about jointly unknown multipliers. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/05-structural.tex>.
- [4] *Forge*, same revision, `prototype/forge/recurrence.py`. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/prototype/forge/recurrence.py>.
- [5] *Forge*, same revision, `article/sections/06-nonlinear.tex`. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/06-nonlinear.tex>.
- [6] *Forge*, same revision, `article/sections/08-integration.tex`. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/08-integration.tex>.
- [7] Vladimir Reshetnikov, *Leant: README*, revision 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/README.md>.
- [8] *Leant*, same revision, `docs/behavioral-synthesis.md`. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/behavioral-synthesis.md>.
- [9] Vladimir Reshetnikov, *Djex: README*, revision e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/README.md>.
- [10] Stefan Kiefer, *Notes on Equivalence and Minimization of Weighted Automata*, 2020, arXiv:2009.01217. <https://arxiv.org/abs/2009.01217>.
- [11] Michael Benedikt, Timothy Duff, Aditya Sharad, and James Worrell, *Polynomial Automata: Zeroness and Applications*, LICS 2017, pp. 1–12. DOI: 10.1109/LICS.2017.8005101. <https://doi.org/10.1109/LICS.2017.8005101>.

- [12] Erdenebayar Bayarmagnai, Fatemeh Mohammadi, and Rémi Prébet, *Algebraic and Algorithmic Methods for Computing Polynomial Loop Invariants*, Journal of Symbolic Computation 135 (2026), 102551; arXiv:2412.14043v2. <https://arxiv.org/html/2412.14043>.
- [13] Hao Shen, Junyu Guo, Junqi Liu, and Lihong Zhi, *Automated Tactics for Polynomial Reasoning in Lean 4*, 2026, arXiv:2604.13514v1. Includes descriptions of `GroebnerTactic`, `MonomialOrderedPolynomial`, `gb_solve`, and ideal-membership certificate replay. <https://arxiv.org/html/2604.13514v1>.
- [14] SymPy development team, *Polynomial Manipulation Reference*. Prototype runtime: SymPy 1.14.0; documentation consulted 15 September 2026. <https://docs.sympy.org/latest/modules/polys/reference.html>.
- [15] Singular development team, *Singular Manual*, `lift` and `liftstd`; documented standard-basis transformation and ideal-membership lifting operations. https://www.singular.uni-kl.de/Manual/4-0-3/sing_334.htm; https://www.singular.uni-kl.de/Manual/4-0-3/sing_335.htm.
- [16] SageMath development team, *Ideals in Multivariate Polynomial Rings*, reference documentation, consulted 15 September 2026. https://doc.sagemath.org/html/en/reference/polynomial_rings/sage/rings/polynomial/multi_polynomial_ideal.html.